



Workpath SDK Event Notification Sample Overview

Product Group: HP Workpath SDK

Version: 1.6.3

Revised: November 11, 2025

HP Inc.
11311 Chinden Blvd.
Boise, ID 83714 USA

HP Confidential

Table of Contents

1	Introduction	3
1.1	Document conventions	3
2	Event Notification Sample on a device	4
3	Actions and permissions	5
3.1	Actions	5
3.2	Permissions	6
4	Event Notification Sample Flow	7
4.1	Broadcast Receivers Flow	7
4.2	MainActivity Flow	9
5	Event Handling	10
5.1	Sign In/Out event handling	10
5.2	Job Completed event handling	12
5.3	Configuration Changed event handling	13
5.4	Sleep/Wake Up event handling	14
6	Legal notice	15
7	Support.....	16

1 Introduction

EventNotificationSample provides a use case scenario on how to receive various events using static broadcast receivers declared in the manifest.

This sample application demonstrates the following key functionalities:

- **Event Detection**: Detect and handle various events.
- **Notification Display**: Display notifications to the user for detected events.

This sample application helps developers understand how to use the HP Workpath SDK to implement event notification features effectively.

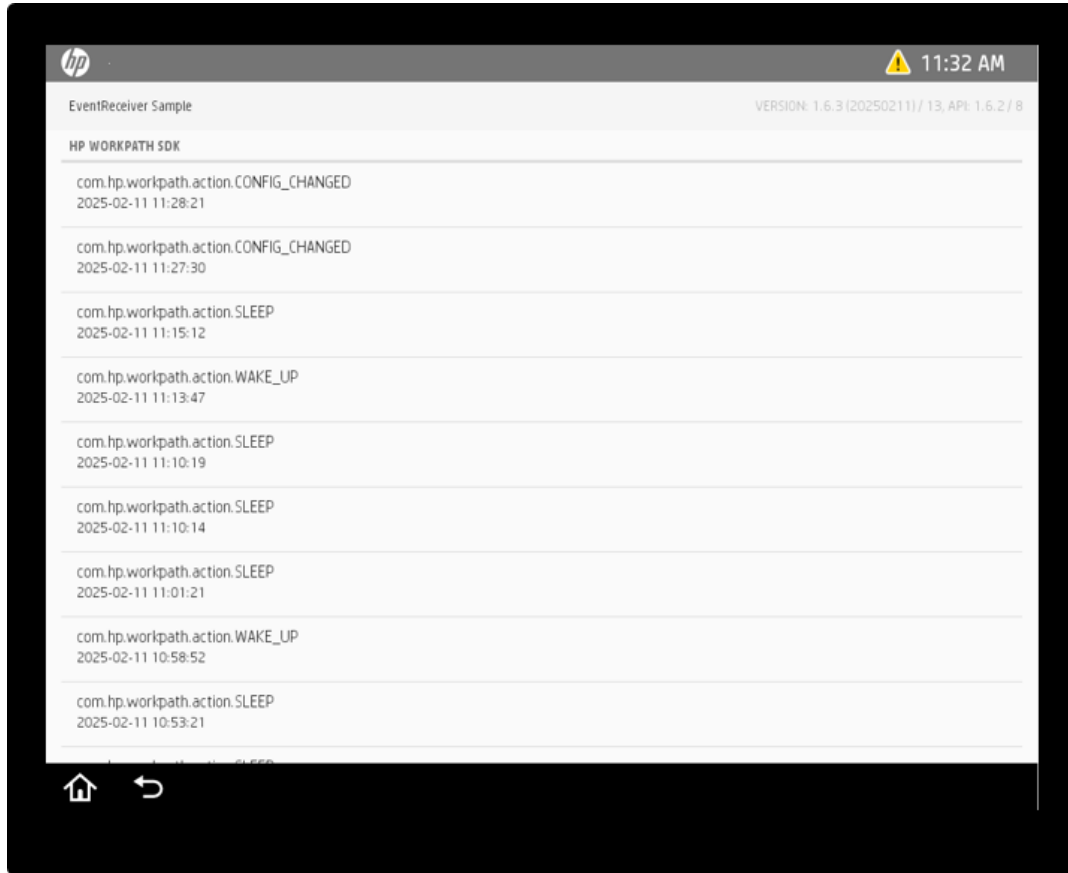
1.1 Document conventions

The following conventions are used throughout this document to define, describe, and discuss the API: function prototypes, data structure definitions, enumerated constants.

- **Name** – Bold signifies item names such as button names and menu items.
- <name> – Angle brackets mark URLs.
- {value} – Curly braces represent values to be replaced.
- Identifiers for defined constants use all UPPERCASE letters.
- *Name* – Italicized type refers to formal function parameters, data structure types and fields, and other defined or enumerated types in the text body (for example, *streamtype*).
- Numbers are in decimal format unless otherwise noted.
- `Name` – Monospaced type is used for code fragments, function prototypes and data type definitions.
- Function names are usually accompanied by parentheses (for example, `function()`).

2 Event Notification Sample on a device

EventNotificationSample demonstrates how to detect events using static broadcast receivers when the HP Workpath SDK sends the events. The latest event will be displayed at the top of the list.



A main activity of Event Notification Sample

3 Actions and permissions

The Workpath SDK provides the following event actions along with their respective permissions.

3.1 Actions

Action	Description
com.hp.workpath.action.SIGN_IN	Action sent as a broadcast intent by the Workpath service to an app with a receiver defined in the manifest when a sign-in event occurs. Required permission: com.hp.workpath.permission.RECEIVE_SIGN_IN_OUT_EVENT
com.hp.workpath.action.SIGN_OUT	Action sent as a broadcast intent by the Workpath service to an app with a receiver defined in the manifest when a sign-out event occurs. Required permission: com.hp.workpath.permission.RECEIVE_SIGN_IN_OUT_EVENT
com.hp.workpath.action.JOB_COMPLETED	Action sent as a broadcast intent by the Workpath service to an app with a receiver defined in the manifest when a walk-up job is completed. Note: This action is triggered when walk-up jobs are completed normally, as well as in cases such as job cancellation or scan failure. However, background print jobs and internal jobs (such as Report generation or Configuration page printing) do not trigger this action. Required permission: com.hp.workpath.permission.RECEIVE_JOB_COMPLETED_EVENT
com.hp.workpath.action.CONFIG_CHANGED	Action sent as a broadcast intent by the Workpath service to an app with a receiver defined in the manifest when the app's configuration changes. Required permission: com.hp.workpath.permission.RECEIVE_CONFIG_CHANGED_EVENT

com.hp.workpath.action. WAKE_UP	Action sent as a broadcast intent by the Workpath service to an app with a receiver defined in the manifest when the device wakes up from sleep mode. Required permission: com.hp.workpath.permission. RECEIVE_SLEEP_WAKEUP_EVENT
com.hp.workpath.action. SLEEP	Action sent as a broadcast intent by the Workpath service to an app with a receiver defined in the manifest when the device wakes up from sleep mode. Required permission: com.hp.workpath.permission. RECEIVE_SLEEP_WAKEUP_EVENT

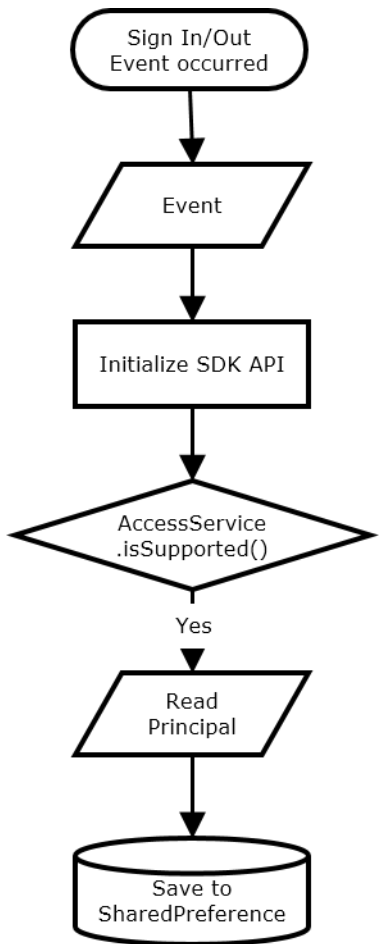
3.2 Permissions

Permission	Description
com.hp.workpath.permission. RECEIVE_SIGN_IN_OUT_EVENT	Permission to receive sign in/out event
com.hp.workpath.permission. RECEIVE_JOB_COMPLETED_EVENT	Permission to receive walk-up job completed event
com.hp.workpath.permission. RECEIVE_CONFIG_CHANGED_EVENT	Permission to receive configuration changed event
com.hp.workpath.permission. RECEIVE_SLEEP_WAKEUP_EVENT	Permission to receive sleep/wake up event

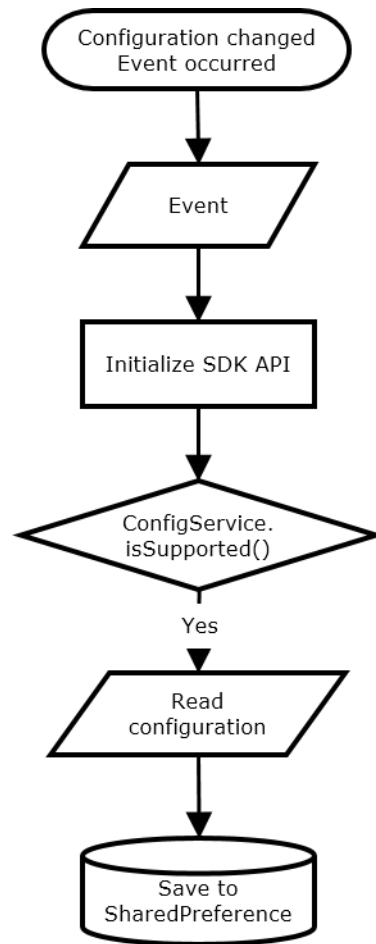
4 Event Notification Sample Flow

The Event Notification Sample is broadly divided into broadcast receivers and MainActivity.

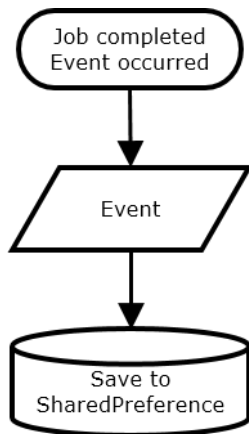
4.1 Broadcast Receivers Flow



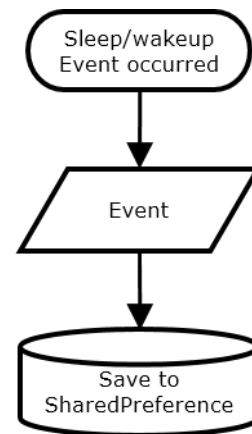
Flow of Sign in/out Receiver



Flow of Configuration Change Receiver

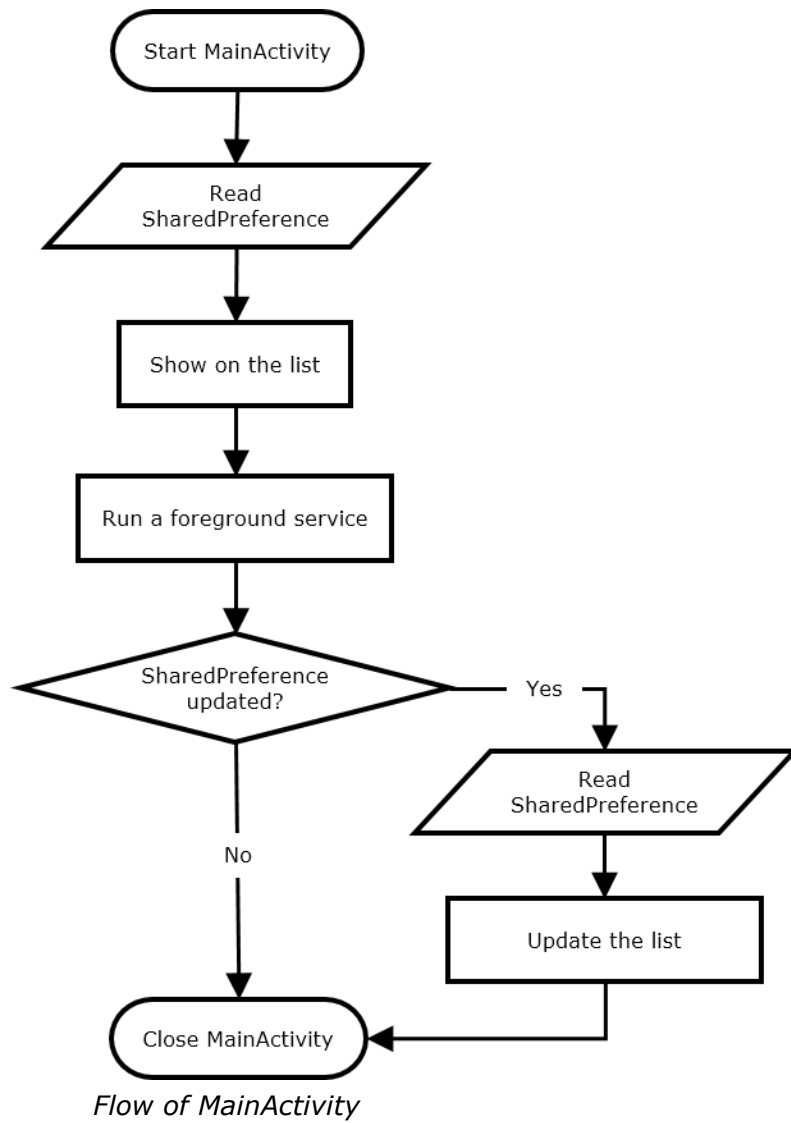


Flow of Job Completed Receiver



Flow of Sleep/Wakeup Receiver

4.2 MainActivity Flow



5 Event Handling

This chapter explains the common aspects of event handling and how to handle each event.

To receive a new event, you must follow the common steps below:

1. Declare uses-permission in AndroidManifest.xml.
2. Declare a broadcast receiver in AndroidManifest.xml with the permission.
3. Implement the broadcast receiver.

(Dynamic broadcast receivers are not supported.)

The following sections describe how to handle specific events.

5.1 Sign In/Out event handling

To handle the Sign In/Out event, first the application must declare the uses-permission and broadcast receiver in the AndroidManifest.xml as follows.

```

1. <uses-permission android:name="com.hp.workpath.permission.RECEIVE_SIGN_IN_OUT_EVENT" />
2. ...
3. <receiver android:name=".receiver.SignInOutReceiver"
4.     android:permission="com.hp.workpath.permission.RECEIVE_SIGN_IN_OUT_EVENT"
5.     android:exported="true" >
6.     <intent-filter>
7.         <action android:name="com.hp.workpath.action.SIGN_IN" />
8.         <action android:name="com.hp.workpath.action.SIGN_OUT" />
9.     </intent-filter>
10. </receiver>

```

Declare uses-permission and broadcast receiver in AndroidManifest.xml

And in the broadcast receiver, you can implement the logic to handle the event as follows.

```

1. @Override
2. public void onReceive(Context context, Intent intent) {
3.     if (Workpath.actions.SIGN_IN.equals(intent.getAction())) {
4.         Log.i(TAG, "Sign in event received");
5.     } else if (Workpath.actions.SIGN_OUT.equals(intent.getAction())) {
6.         Log.i(TAG, "Sign out event received");
7.     }
8. }

```

Implementation in the broadcast receiver (SignInOutReceiver)

The broadcast receiver in this sample receives sign in/out events and retrieves the principal of the currently active user to store it in SharedPreferences.

One important point to note is that to prevent the process from being killed while handling the sign in/out event, the MainActivity should start a foreground service. If a reset occurs after a sign in/out event and MainActivity has been previously executed, it will be included in the recent tasks list so the process will be killed by the reset. Then the background broadcast receiver may not complete its task because the process is terminated. Starting a foreground service in MainActivity will keep the process alive and prevent such problems. There are several ways to start a foreground service, but in this sample, it is done as follows.

```

1. <service
2.     android:name=".service.ForegroundService"
3.     android:enabled="true"
4.     android:exported="false" />

```

Declare a foreground service in AndroidManifest.xml

```

1. public class ForegroundService extends Service {
2.     private static final String TAG = MainActivity.TAG + "S";
3.     private int ACCESS_NOTIFICATION_ID = 2024;
4.
5.     @Override
6.     public void onCreate() {
7.         super.onCreate();
8.         createNotificationChannel();
9.         Log.i(TAG, "onCreate");
10.    }
11.
12.    private void createNotificationChannel() {
13.        String NOTIFICATION_CHANNEL_ID = getString(R.string.app_name);
14.        String channelName = getString(R.string.channel_name);
15.        NotificationChannel nc = new NotificationChannel(NOTIFICATION_CHANNEL_ID,
channelName, NotificationManager.IMPORTANCE_MIN);
16.        nc.setLockscreenVisibility(Notification.VISIBILITY_PRIVATE);
17.        NotificationManager manager = (NotificationManager)
getSystemService(Context.NOTIFICATION_SERVICE);
18.        manager.createNotificationChannel(nc);
19.        NotificationCompat.Builder notificationBuilder = new
NotificationCompat.Builder(this, NOTIFICATION_CHANNEL_ID);
20.        notificationBuilder.setForegroundServiceBehavior(NotificationCompat.FOREGROUND_SERVICE_
IMMEDIATE);
21.        notificationBuilder.setPriority(NotificationCompat.PRIORITY_MAX);
22.        Notification notification = notificationBuilder.setOngoing(true)
23.            .setSmallIcon(R.drawable.ic_transparent)
24.            .setContentTitle(getText(R.string.channel_description))
25.            .setPriority(NotificationManager.IMPORTANCE_MAX)
26.            .setCategory(Notification.CATEGORY_SERVICE)
27.            .build();
28.        startForeground(ACCESS_NOTIFICATION_ID, notification,
ServiceInfo.FOREGROUND_SERVICE_TYPE_MANIFEST);
29.    }
30. }

```

Implement the foreground service

```

1. public class MainActivity extends AppCompatActivity {
2.     ...
3.     @Override
4.     public void onCreate() {
5.         super.onCreate();
6.         ...
7.         Intent startServiceIntent = new Intent(getApplicationContext(),
8.             ForegroundService.class);
9.         getApplicationContext().startService(startServiceIntent);
10.    }
11. }

```

Start the foreground service in MainActivity

5.2 Job Completed event handling

To handle the Job Completed event, first the application must declare the uses-permission and broadcast receiver in the AndroidManifest.xml as follows.

```

1. <uses-permission android:name="com.hp.workpath.permission.RECEIVE_JOB_COMPLETED_EVENT"
2.     />
3.     ...
4. <receiver android:name=".receiver.JobCompletedReceiver"
5.     android:permission="com.hp.workpath.permission.RECEIVE_JOB_COMPLETED_EVENT"
6.     android:exported="true" >
7.     <intent-filter>
8.         <action android:name="com.hp.workpath.action.JOB_COMPLETED" />
9.     </intent-filter>
10. </receiver>

```

Declare uses-permission and broadcast receiver in AndroidManifest.xml

And in the broadcast receiver, you can implement the logic to handle the event as follows.

```

1. @Override
2. public void onReceive(Context context, Intent intent) {
3.     Log.i(TAG, "Job completed event received");
4.     PreferenceManager.saveEvent(context, intent.getAction(), null);
5. }

```

Implementation in the broadcast receiver (JobCompletedReceiver)

This sample demonstrates receiving Job Completed events and saving the event on the SharedPreferences.

5.3 Configuration Changed event handling

To handle the Configuration Changed event, first the application must declare the uses-permission and broadcast receiver in the AndroidManifest.xml as follows.

```

1. <uses-permission android:name="com.hp.workpath.permission.RECEIVE_CONFIG_CHANGED_EVENT"
   />
2. ...
3. <receiver android:name=".receiver.ConfigChangedReceiver"
4.     android:permission="com.hp.workpath.permission.RECEIVE_CONFIG_CHANGED_EVENT"
5.     android:exported="true" >
6.     <intent-filter>
7.         <action android:name="com.hp.workpath.action.CONFIG_CHANGED" />
8.     </intent-filter>
9. </receiver>

```

Declare uses-permission and broadcast receiver in AndroidManifest.xml

And in the broadcast receiver, you can implement the logic to handle the event as follows.

```

1. @Override
2. public void onReceive(Context context, Intent intent) {
3.     Log.i(TAG, "Config changed event received");
4.     mContextRef = new WeakReference<>(context);
5.     InitializationTask initTask = new ConfigInitializationTask(context, this);
6.     initTask.taskExecute();
7.     initTask.cancel();
8. }
9. ...
10. @Override
11. public void handleUpdate(String updateData) {
12.     PreferenceManager.saveEvent(mContextRef.get(), Workpath.actions.CONFIG_CHANGED,
        updateData);
13. }

```

Implementation in the broadcast receiver (ConfigChangedReceiver)

The CONFIG_CHANGED event is delivered only to the receiver registered by the application package whose configuration has changed, and other packages are not aware of it.

This sample demonstrates using ConfigService to retrieve the changed configuration and save it in SharedPreferences. The InitializationTask is also covered in ConfigSample, so it is omitted here. If you are curious, please refer to the InitializationTask and ConfigInitializationTask in EventNotificationSample.

5.4 Sleep/Wake Up event handling

To handle the Sleep/Wake Up event, first the application must declare the uses-permission and broadcast receiver in the AndroidManifest.xml as follows.

```
1. <uses-permission android:name="com.hp.workpath.permission.RECEIVE_SLEEP_WAKEUP_EVENT"
   />
2. ...
3. <receiver android:name=".receiver.SleepWakeUpReceiver"
4.     android:permission="com.hp.workpath.permission.RECEIVE_SLEEP_WAKEUP_EVENT"
5.     android:exported="true" >
6.     <intent-filter>
7.         <action android:name="com.hp.workpath.action.WAKE_UP" />
8.         <action android:name="com.hp.workpath.action.SLEEP" />
9.     </intent-filter>
10. </receiver>
```

Declare uses-permission and broadcast receiver in AndroidManifest.xml

And in the broadcast receiver, you can implement the logic to handle the event as follows.

```
1. @Override
2. public void onReceive(Context context, Intent intent) {
3.     Log.i(TAG, "Sleep/Wakeup event received");
4.     PreferenceManager.saveEvent(context, intent.getAction(), null);
5. }
```

Implementation in the broadcast receiver (SleepWakeUpReceiver)

This sample demonstrates receiving Sleep/Wake Up Completed events and saving the event on the SharedPreferences.

6 Legal notice

The information contained in this documentation is subject to change without notice.

HP Inc. makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. HP Inc. shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance, or use of this material.

This document contains proprietary information that is protected by copyright. All rights are reserved. No part of this documentation may be transformed, reproduced, or translated to another language without prior.

© Copyright 2018 HP Development Company, L.P.

7 Support

The HP Solution Business Program (SBP) support staff has established a procedure for submitting technical problems to the Incident Call Center (ICC). All SBP members must be registered with the Solution Programs Portal (SPP) to access technical support and other feedback or reporting activities.

Log on to the portal at <<http://www.hp.com/go/solutions>> to submit your issue to technical support. After you submit the issue, a report will be generated for the Incident Call Center. A Technical Support representative will be assigned to validate the incident and to work on a fix. HP will contact you for further information regarding the incident as quickly as possible.